

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ПОЛТАВСЬКА ПОЛІТЕХНІКА
ІМЕНІ ЮРІЯ КОНДРАТЮКА»

Навчально науковий інститут інформаційних технологій і робототехніки
Кафедра комп'ютерних та інформаційних технологій і систем

Лабораторна робота № 4

з навчальної дисципліни

«Технології розробки корпоративних Web-додатків»

Виконав:

Студент групи 403-ТН

Солонецький Роман Миколайович

Перевірив:

Мізік Андрій Олегович

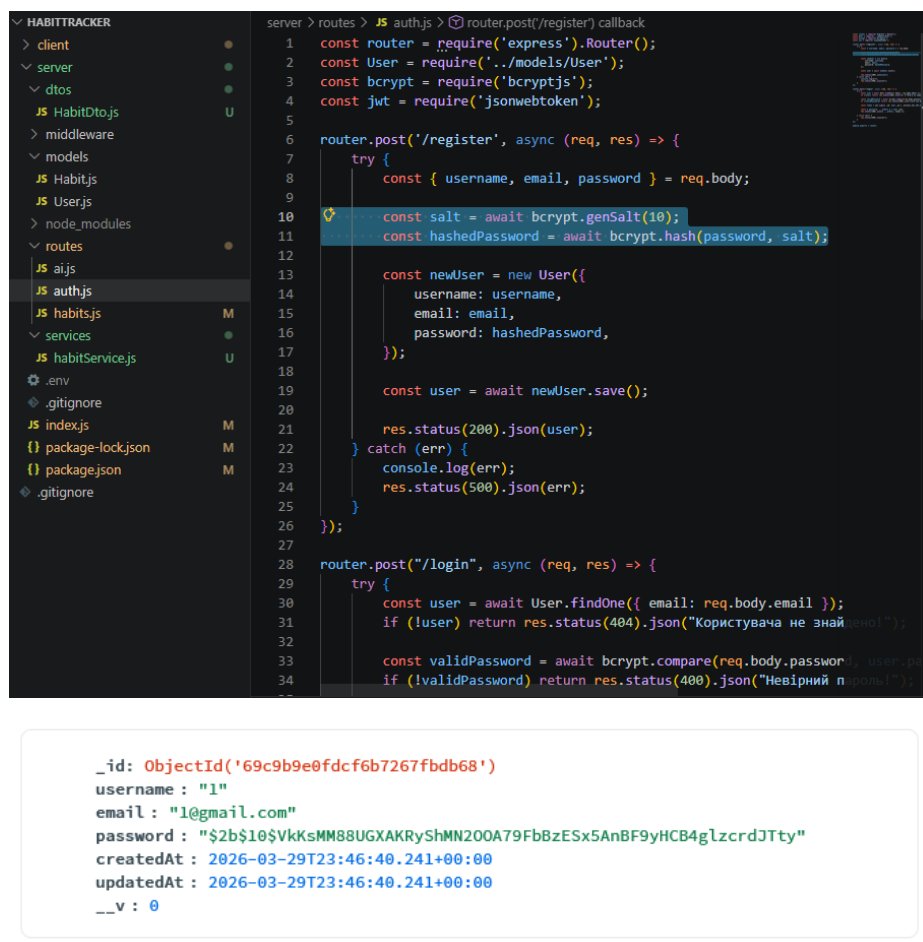
Полтава 2026

Тема: Вебдодаток для відстеження звичок (HabitTracker)

Для звіту використовую вже майже готовий проект до дипломної роботи.

Хешування паролів та реєстрація

Для реєстрації користувачів реалізовано механізм хешування паролів за допомогою бібліотеки `bcryptjs`. Під час реєстрації створюється унікальна «сіль», яка додається до пароля перед генерацією хешу. Це унеможливляє відновлення пароля навіть у разі отримання доступу до бази даних (захист від атак по таблицях райдуги). У базі даних MongoDB зберігається лише результат хешування.



```
server > routes > JS authjs > router.post('/register') callback
1  const router = require('express').Router();
2  const User = require('../models/User');
3  const bcrypt = require('bcryptjs');
4  const jwt = require('jsonwebtoken');
5
6  router.post('/register', async (req, res) => {
7    try {
8      const { username, email, password } = req.body;
9
10     const salt = await bcrypt.genSalt(10);
11     const hashedPassword = await bcrypt.hash(password, salt);
12
13     const newUser = new User({
14       username: username,
15       email: email,
16       password: hashedPassword,
17     });
18
19     const user = await newUser.save();
20
21     res.status(200).json(user);
22   } catch (err) {
23     console.log(err);
24     res.status(500).json(err);
25   }
26 });
27
28 router.post('/login', async (req, res) => {
29   try {
30     const user = await User.findOne({ email: req.body.email });
31     if (!user) return res.status(404).json("Користувача не знайдено!");
32
33     const validPassword = await bcrypt.compare(req.body.password, user.password);
34     if (!validPassword) return res.status(400).json("Невірний пароль!");
35   }
36 });
```

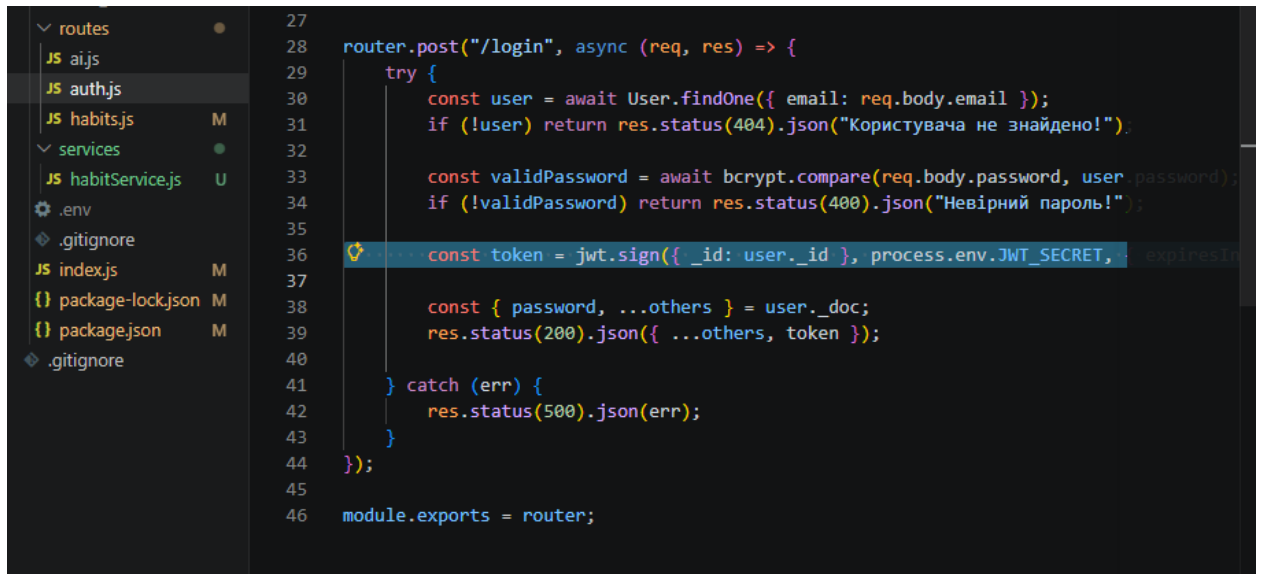
```
_id: ObjectId('69c9b9e0fdcf6b7267fbd68')
username: "1"
email: "1@gmail.com"
password: "$2b$10$VvKsMM88UGXAKRyShMN200A79FbBzESx5AnBF9yHCB4gIzcrdJTty"
createdAt: 2026-03-29T23:46:40.241+00:00
updatedAt: 2026-03-29T23:46:40.241+00:00
__v: 0
```

Рис 1-2 – Реалізація бібліотеки `bcryptjs` та запис користувача в БД

Реалізація логіну та генерація токена (JWT)

Процедура авторизації (Login) включає перевірку існування користувача та порівняння введеного пароля з хешем у БД. Після успішної автентифікації сервер генерує JSON Web Token (JWT), який підписується

секретним ключем із середовища оточення (process.env.TOKEN_SECRET). Токен містить ідентифікатор користувача (_id) та передається клієнту для подальшої ідентифікації запитів.



```
27
28 router.post("/login", async (req, res) => {
29   try {
30     const user = await User.findOne({ email: req.body.email });
31     if (!user) return res.status(404).json("Користувача не знайдено!");
32
33     const validPassword = await bcrypt.compare(req.body.password, user.password);
34     if (!validPassword) return res.status(400).json("Невірний пароль!");
35
36     const token = jwt.sign({ _id: user._id }, process.env.JWT_SECRET, { expiresIn: '1h' });
37
38     const { password, ...others } = user._doc;
39     res.status(200).json({ ...others, token });
40
41   } catch (err) {
42     res.status(500).json(err);
43   }
44 });
45
46 module.exports = router;
```

Рис 3 – Реалізація авторизації

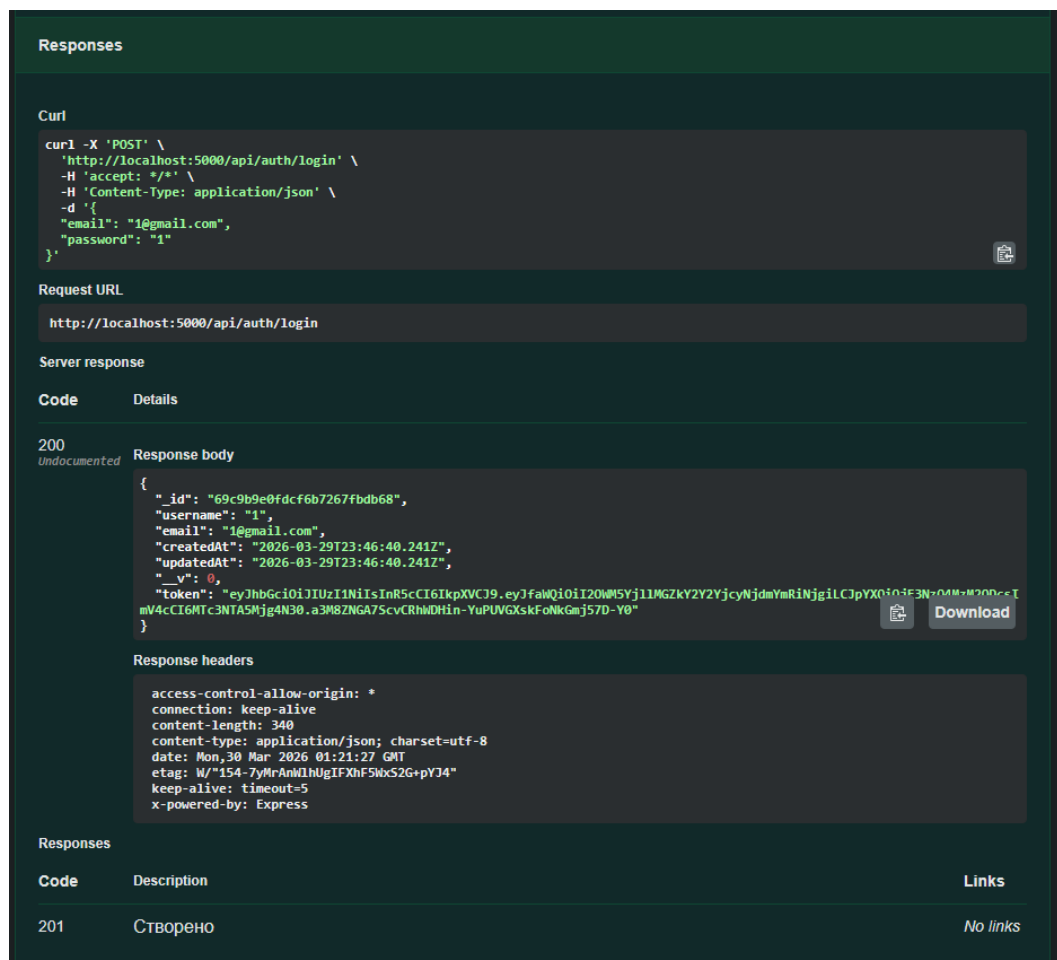
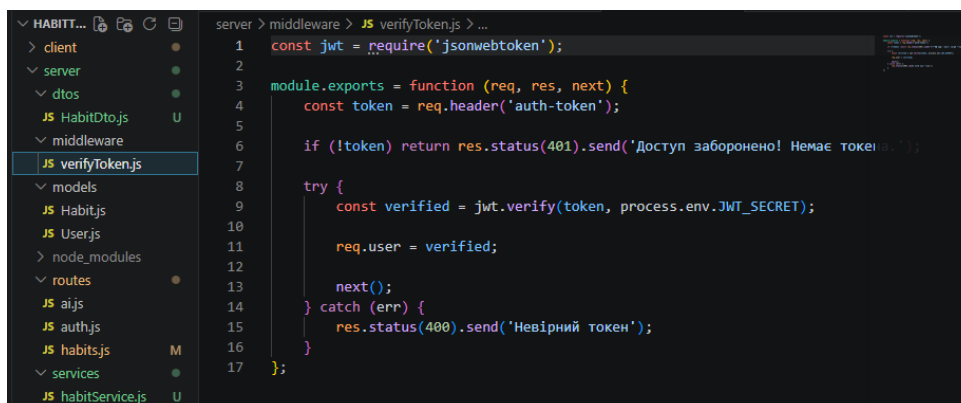


Рис 4 – Відповідь сервера на POST /api/auth/login при пошуку користувача у Swagger

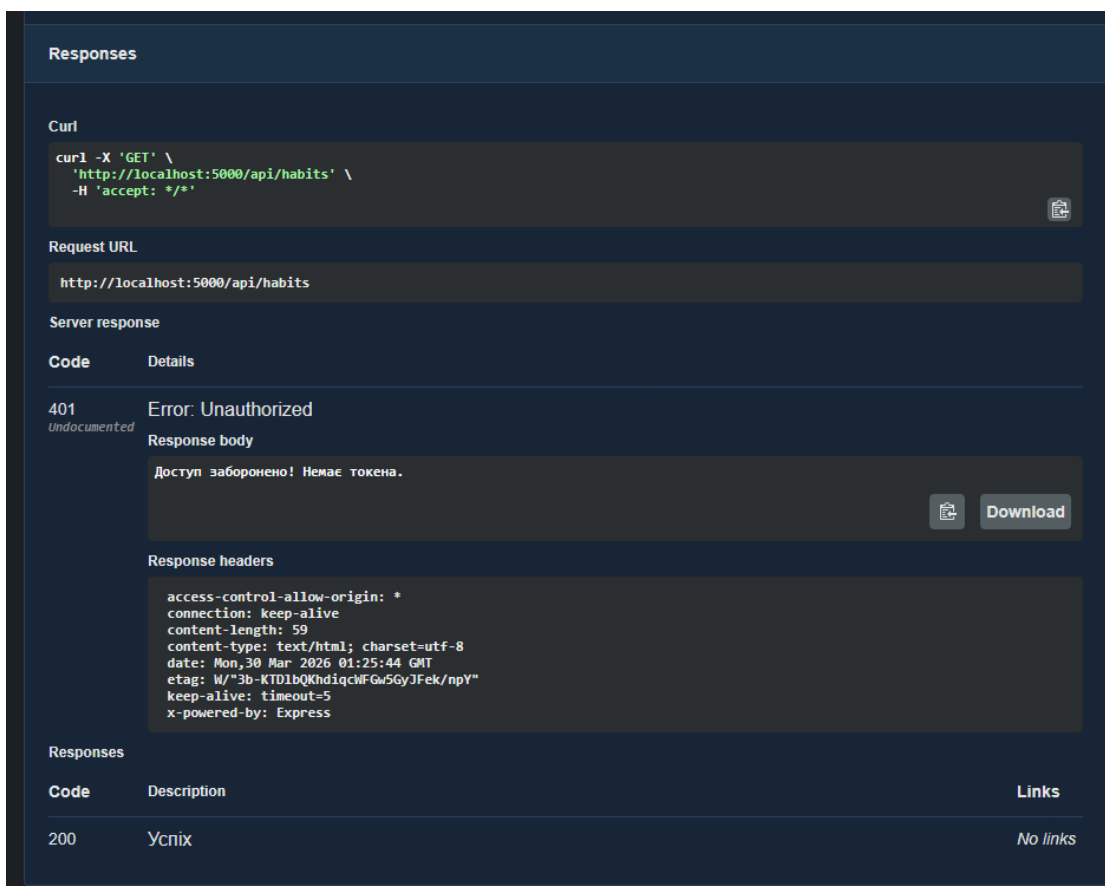
Middleware для перевірки токена

Захист приватних маршрутів реалізовано через Middleware функцію. Вона перехоплює HTTP-запит, витягує токен із заголовка auth-token (або Authorization), верифікує його підпис і, у разі успіху, додає дані користувача до об'єкта запиту (req.user). Якщо токен відсутній або невалідний, сервер повертає статус 401 Unauthorized.



```
server > middleware > JS verifyToken.js > ...
1  const jwt = require('jsonwebtoken');
2
3  module.exports = function (req, res, next) {
4    const token = req.header('auth-token');
5
6    if (!token) return res.status(401).send('Доступ заборонено! Немає токена.');
```

Рис 5 – Реалізація Middleware функції



Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5000/api/habits' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:5000/api/habits
```

Server response

Code	Details
401	Error: Unauthorized

Response body

```
Доступ заборонено! Немає токена.
```

Response headers

```
access-control-allow-origin: *
connection: keep-alive
content-length: 59
content-type: text/html; charset=utf-8
date: Mon, 30 Mar 2026 01:25:44 GMT
etag: W/"3b-KTD1bQKhdicqWfGw5GyJFek/npY"
keep-alive: timeout=5
x-powered-by: Express
```

Responses

Code	Description	Links
200	Успіх	No links

Рис 6 – Відповідь сервера на GET без авторизації у Swagger

Реалізація перевірки ролей (RBAC, 403 Forbidden)

Для управління доступом впроваджено модель RBAC (Role-Based Access Control). Кожному користувачу присвоюється роль (user або admin). Спеціальне Middleware перевіряє рівень доступу перед виконанням операції. Якщо роль користувача не відповідає вимогам маршруту (наприклад, видалення системних даних), сервер повертає статус 403 Forbidden.

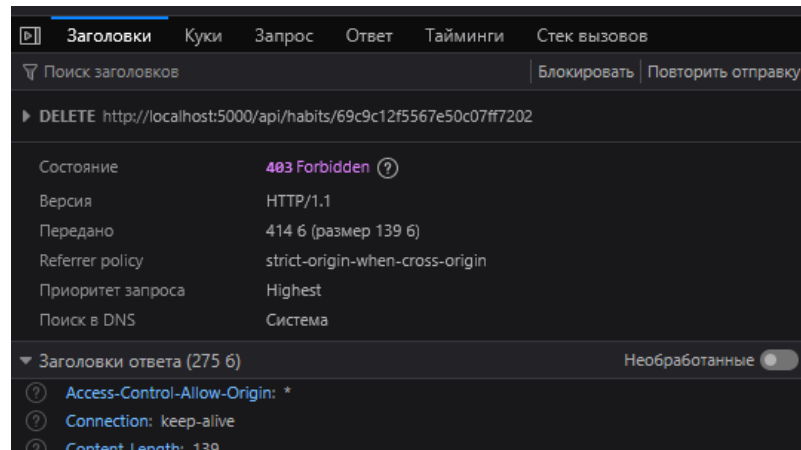


Рис 7 – Запит користувача на видалення звички, видає статус 403

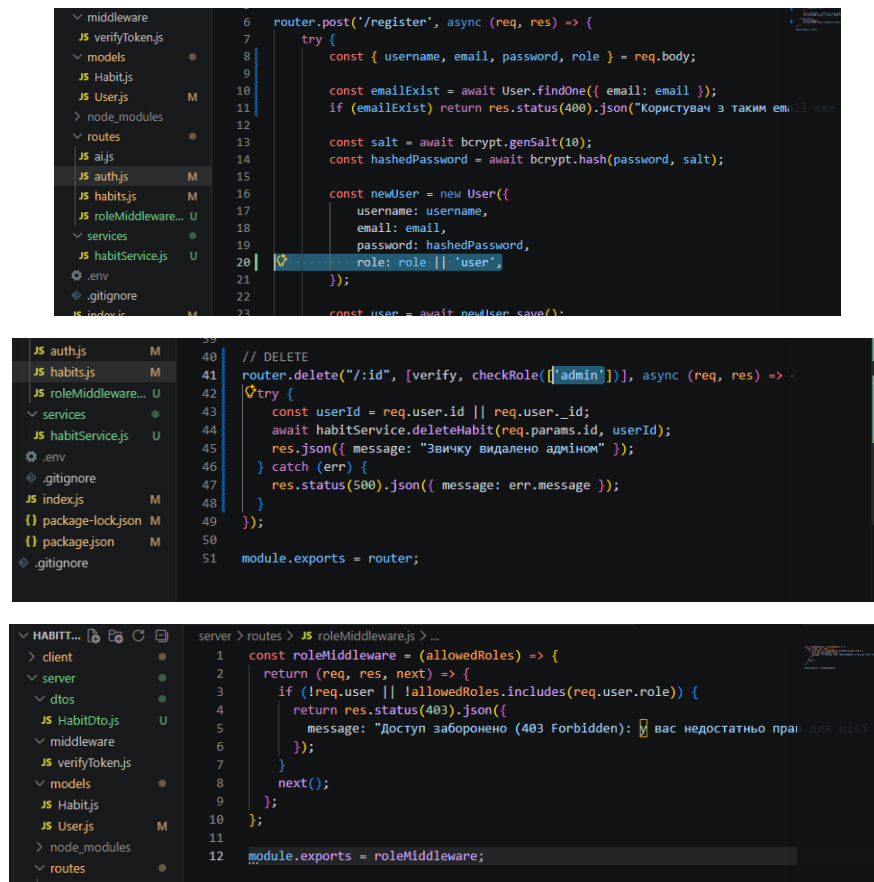


Рис 8-9 – Реалізація видачі ролі при реєстрації та перевірка при запиті видалення звички

Контрольні питання

1. У чому різниця між автентифікацією (Authentication) та авторизацією (Authorization)?

Автентифікація - це процес перевірки особистості користувача, зазвичай через логін та пароль. Авторизація - це перевірка прав доступу вже автентифікованого користувача, наприклад, доступ до адмін-панелі.

2. Чому не можна зберігати паролі у відкритому вигляді? Що таке "Сіль" (Salt) і навіщо вона потрібна при хешуванні?

Паролі не можна зберігати у відкритому вигляді, бо у разі витоку бази даних зломисники отримають доступ до всіх акаунтів. Сіль - це випадковий рядок даних, що додається до пароля перед хешуванням. Вона потрібна для того, щоб навіть однакові паролі різних користувачів мали різні хеші, що захищає від злому за допомогою заздалегідь обчислених таблиць.

3. Опишіть структуру JWT токена (Header, Payload, Signature). Де зберігається інформація про роль користувача?

Header (Заголовок): містить тип токена та алгоритм шифрування.

Payload (Корисне навантаження): містить дані користувача (id, ім'я, термін дії). Саме тут зазвичай зберігається інформація про роль користувача.

Signature (Підпис): створюється шляхом хешування заголовка та корисного навантаження з секретним ключем для перевірки цілісності токена.

4. Яка різниця між симетричним та асиметричним шифруванням при підписі токенів?

При симетричному шифруванні для створення і перевірки підпису використовується один і той самий секретний ключ. При асиметричному шифруванні використовується пара ключів: приватний (для створення підпису на сервері) та публічний (для перевірки підпису будь-яким сервісом).

5. Що таке атаки Brute Force та Rainbow Table? Як захиститися від них?

Brute Force - це метод підбору пароля шляхом перебору всіх можливих комбінацій символів.

Rainbow Table - це атака з використанням величезних таблиць заздалегідь обчислених хешів для швидкого пошуку пароля за його хешем. Захист: використання довгих паролів, хешування з сіллю та обмеження кількості спроб входу .

6. Поясніть принцип роботи Role-Based Access Control (RBAC).

Role-Based Access Control - це модель управління доступом, де права надаються не конкретним користувачам, а ролям. Користувач отримує доступ до ресурсів автоматично, залежно від того, яка роль йому призначена в системі.